DESPLIEGUE DE APLICACIONES WEB — CURSO 2026/2027

Imágenes de contenedores

Tema 2 — Virtualización y contenedores



Objetivo de la sesión

Una imagen funcional que podemos mejorar y comparar

Punto de partida

Una aplicación Java que necesita compilarse.

Una imagen sencilla con Maven, JDK y todo el proyecto.

Resultado de la sesión

Menos trabajo repetido al construir.

Solo lo necesario para ejecutar.

Usuario y almacenamiento preparados.

Cada mejora debe responder a un problema concreto de construcción o ejecución.

Construcción y arranque del contenedor

Durante la construcción

FROM: elige la base.

WORKDIR: fija el directorio.

COPY: incorpora ficheros.

RUN: ejecuta órdenes de build.

Configuración de ejecución

USER: selecciona el usuario.

EXPOSE: documenta el puerto.

ENTRYPOINT y CMD: definen el comando de arranque.

RUN actúa durante el build. ENTRYPOINT y CMD describen el arranque del contenedor.

Ejemplo: aplicación Java con Maven

```
FROM maven:3.9.16-eclipse-temurin-21-alpine
WORKDIR /app

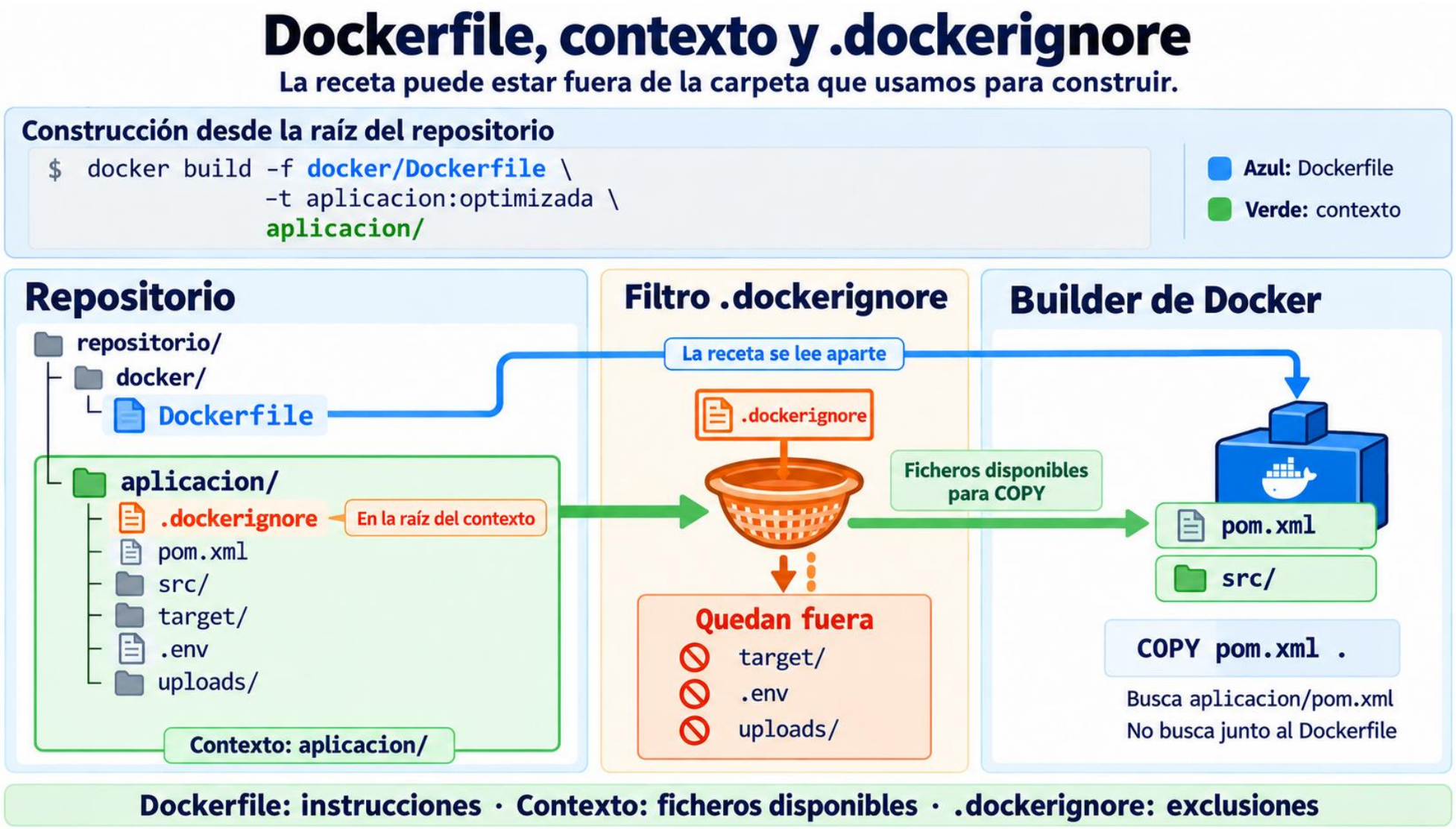
COPY . .
RUN mvn -B package -DskipTests

ENTRYPOINT ["java", "-jar", "target/<artefacto>.war"]
```

Maven, el JDK y el código fuente permanecen en la imagen.
<artefacto> debe sustituirse por el nombre real del WAR generado.

Ejemplo: WAR ejecutable de Spring Boot. No todos los WAR admiten java -jar.

Dockerfile y contexto de construcción



Exclusiones del contexto con .dockerignore

aplicacion/.dockerignore

```
target/
.idea/
.vscode/
*.iml

.env
.env.*

uploads/
```

Menos ficheros irrelevantes para construir.

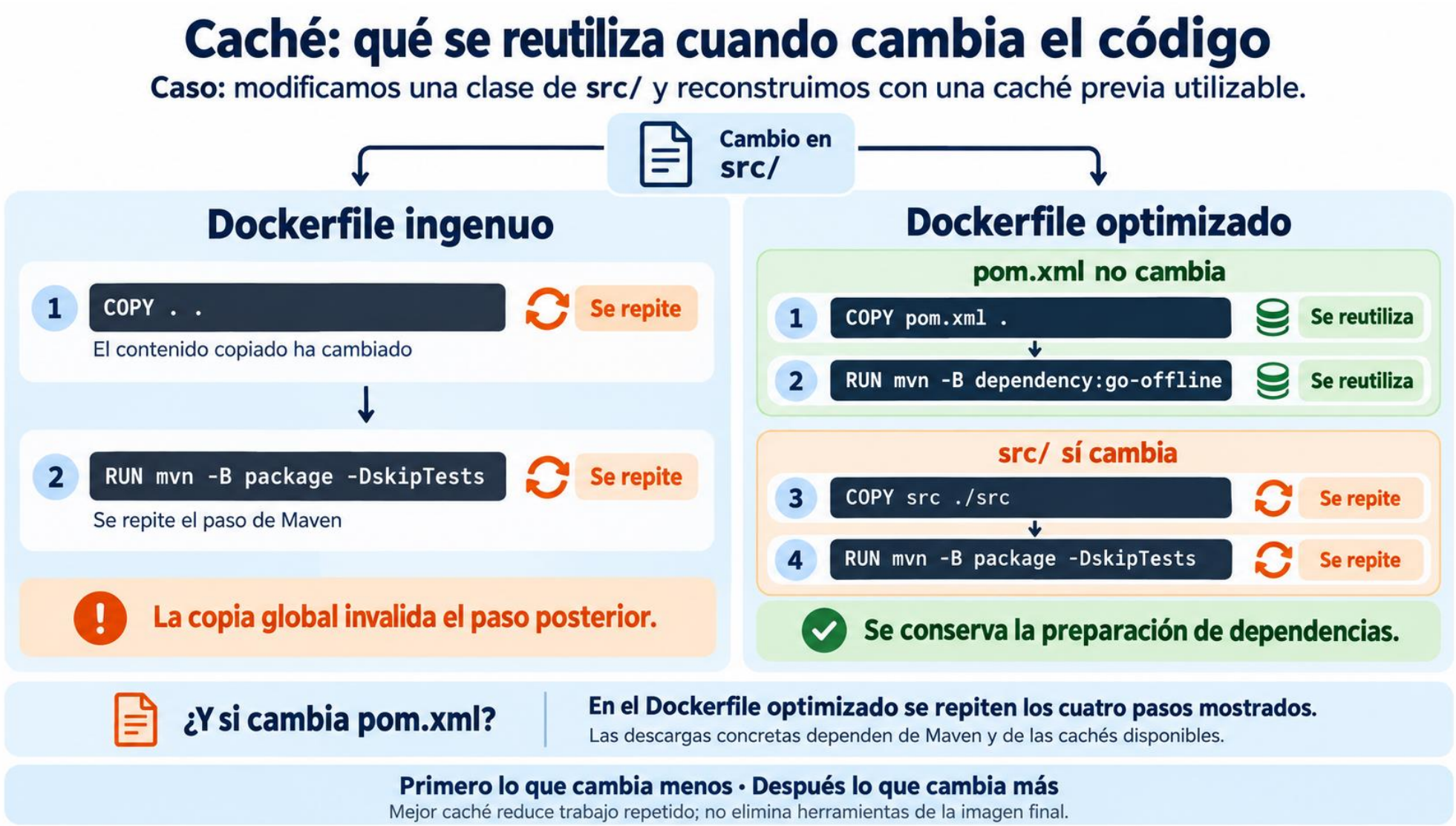
Menos cambios ajenos al código que afectan al build.

Menor riesgo de incorporar datos locales o sensibles.

.gitignore actúa sobre nuevas incorporaciones a Git.
.dockerignore filtra el contexto de Docker.

El contexto define qué puede utilizar COPY, no qué entra automáticamente en la imagen.

Reutilización de pasos de construcción



Medición del tiempo y del tamaño

Tiempo de construcción

Comparar el build inicial y la reconstrucción tras cambiar código.

Anotar si había imágenes base descargadas y caché previa.

```
time docker build ...
```

Tamaño de imagen

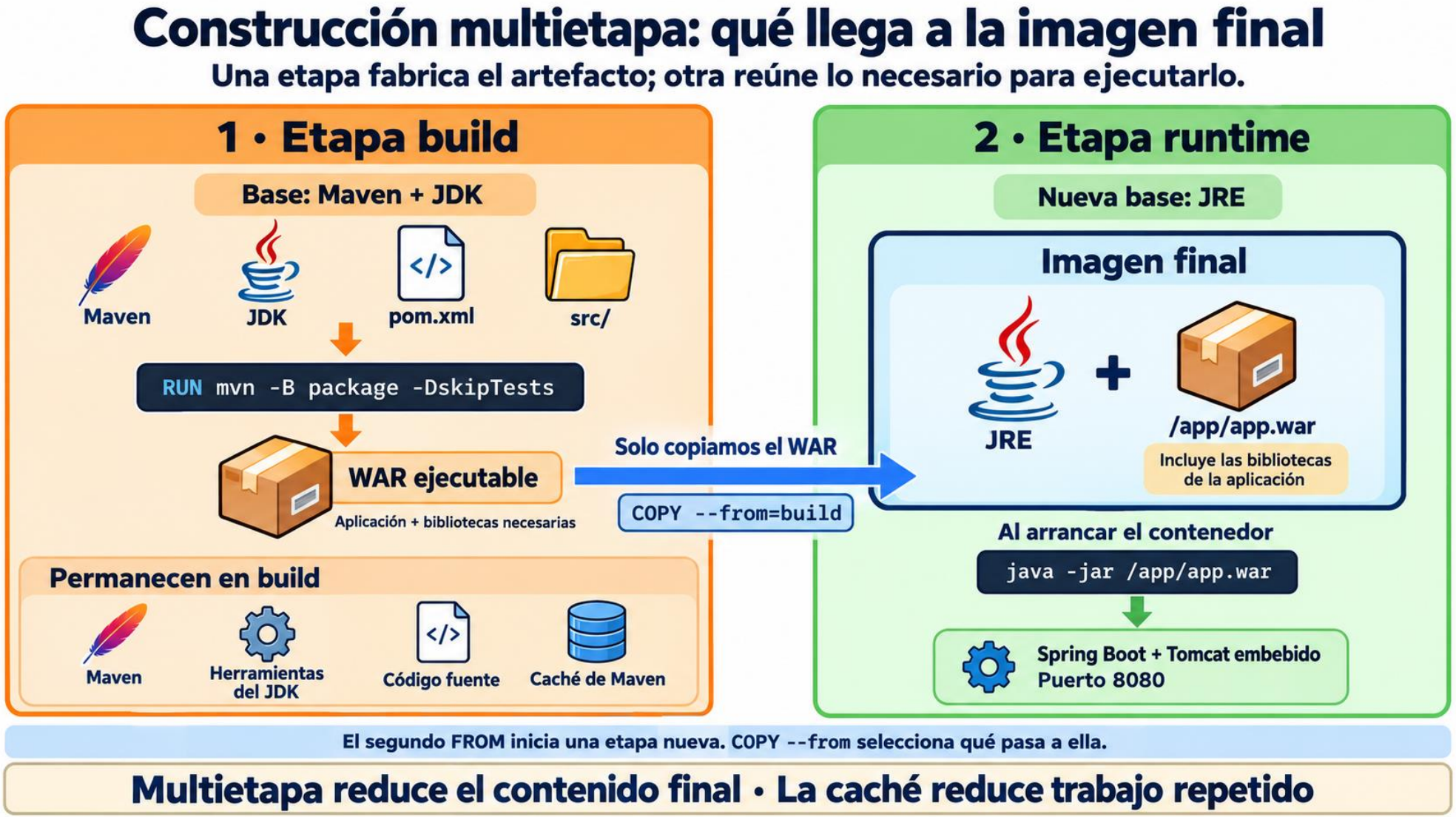
Comparar la columna SIZE con la misma herramienta.

docker system df mide el uso global, incluidas cachés y capas compartidas.

```
docker image ls aplicacion:ingenua
docker image ls aplicacion:optimizada
```

La caché mejora reconstrucciones. No elimina Maven ni el código de la imagen final.

Construcción multietapa



Dockerfile multietapa: construcción

```
FROM maven:3.9.16-eclipse-temurin-21-alpine AS build
WORKDIR /app

COPY pom.xml .
RUN mvn -B dependency:go-offline

COPY src ./src
RUN mvn -B package -DskipTests \
    && cp target/*.war /tmp/app.war
```

El artefacto queda disponible como /tmp/app.war.

Las dependencias se preparan antes de copiar src para aprovechar la caché.

Dockerfile multietapa: ejecución

```
FROM eclipse-temurin:21.0.12_8-jre-alpine-3.24
WORKDIR /app

COPY --from=build /tmp/app.war /app/app.war

EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/app.war"]
```

El segundo FROM inicia una etapa con otra base.
COPY --from=build selecciona qué recibe de la etapa anterior.

La etapa final incorpora el WAR y conserva sus bibliotecas necesarias para ejecutar.

Usuario y almacenamiento de la aplicación

Usuario app y carpeta escribible

La aplicación necesita permisos sobre las rutas que utiliza.

Ejemplo: aplicación Java que interpreta APP_STORAGE_PATH

1 Preparación en el Dockerfile

Durante la construcción de la imagen

1. **Crear el usuario y su grupo**

```
RUN addgroup -S app \
   && adduser -S app -G app
```

2. **Preparar la carpeta y su propietario**

```
RUN mkdir -p /data/uploads \
  && chown -R app:app /data
```

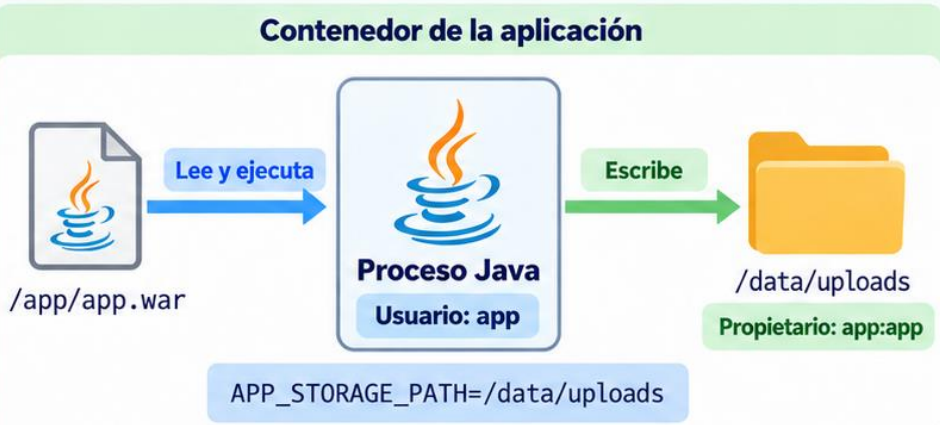
3. **Indicar la ruta a la aplicación**

```
ENV APP_STORAGE_PATH=/data/uploads
```

4. **Seleccionar el usuario de ejecución**

```
USER app
```

2 Ejecución en el contenedor



Comprobaciones

`id`	Usuario distinto de root.
`test -w "$APP_STORAGE_PATH"`	La ruta permite escritura.

Escritura y persistencia son decisiones distintas

Crear /data/uploads no crea un volumen. Sin un montaje persistente, sus datos se pierden al eliminar el contenedor.

Usuario adecuado + ruta configurada + permisos de escritura

Elección y mantenimiento del runtime

Opciones de imagen base

Completa: más herramientas y más software.

Alpine o slim: menor tamaño habitual, con posibles ausencias.

Distroless: diagnóstico interactivo más difícil.

Decisiones del módulo

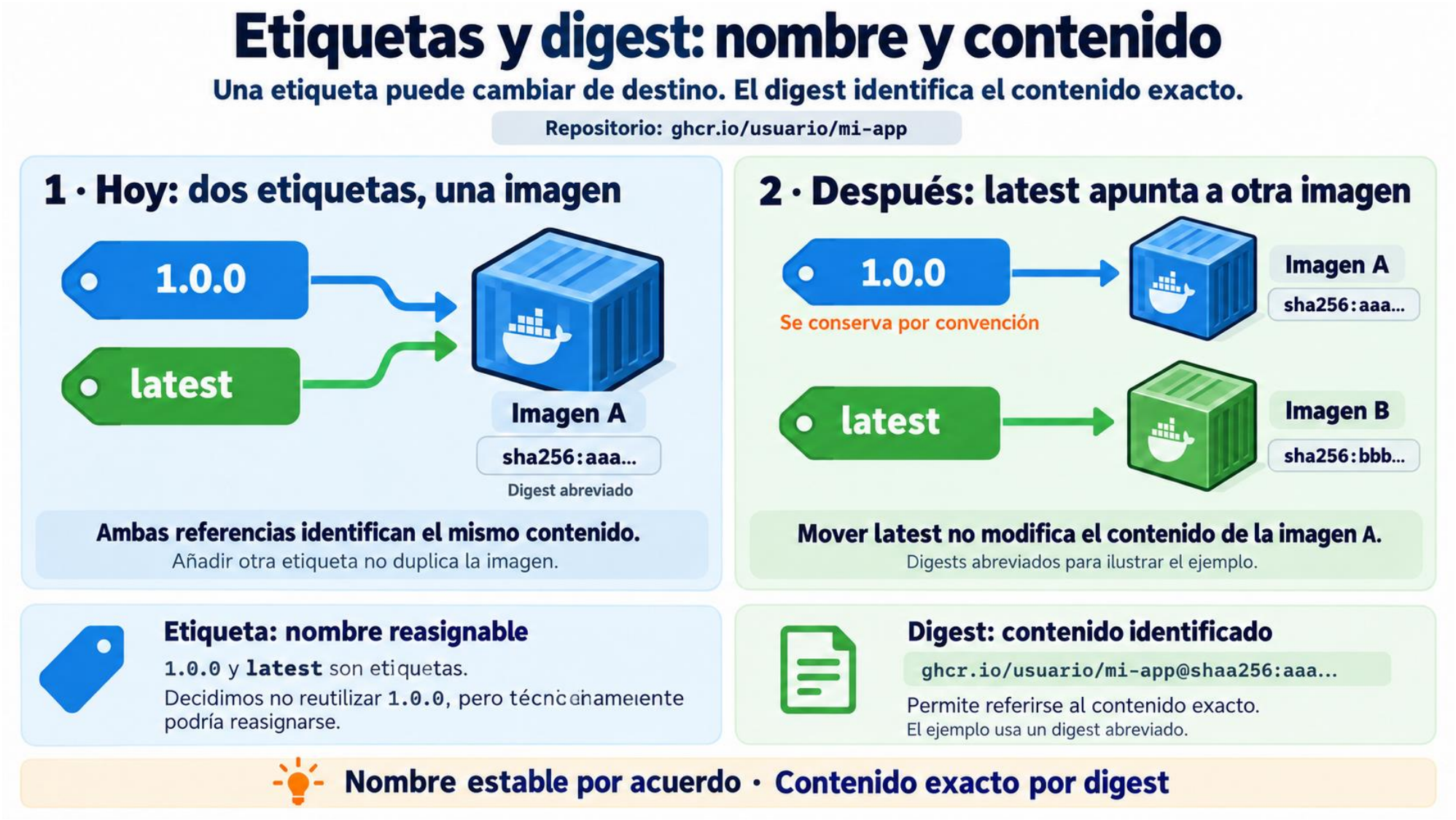
Usaremos variantes mínimas que permitan inspeccionar el contenedor.

Las imágenes base también reciben correcciones.

El mantenimiento exige reconstruir y actualizar periódicamente.

Una imagen pequeña debe seguir siendo adecuada para la aplicación y su diagnóstico.

Identificación mediante etiquetas y digest



Publicación de la imagen en GHCR

```
docker login ghcr.io -u <usuario>

docker tag aplicacion:optimizada \
  ghcr.io/<usuario>/mi-app:1.0.0
docker push ghcr.io/<usuario>/mi-app:1.0.0

docker tag aplicacion:optimizada \
  ghcr.io/<usuario>/mi-app:latest
docker push ghcr.io/<usuario>/mi-app:latest
```

Las dos etiquetas pueden apuntar al mismo digest sin duplicar el contenido.

Validación automática de la construcción

Antes de publicar

Comprobar la estructura del proyecto.

Construir la imagen desde el repositorio.

Ejecutar las pruebas adecuadas.

Qué verificar

Contexto y rutas correctos.

Ausencia de archivos locales imprescindibles.

Arranque y permisos de la aplicación.

Un entorno limpio permite detectar dependencias que solo existían en tu equipo.

Conexión con la actividad 2.2

Construcción y comparación

Identificar qué necesita la aplicación.

Construir una imagen inicial.

Optimizar y comparar tiempo y tamaño.

Verificación y publicación

Comprobar usuario y escritura.

Publicar las etiquetas de la actividad.

Documentar resultados y validar la construcción en CI.

Aplicaremos estas decisiones al proyecto Escaparate.

Cierre de la sesión

Cada decisión resuelve un problema diferente

.dockerignore reduce contenido innecesario del contexto.

La caché permite reutilizar trabajo de construcción.

Multietapa deja las herramientas de build fuera de la imagen final.

El usuario y los permisos controlan la escritura durante la ejecución.

Las etiquetas nombran imágenes y el digest identifica contenido exacto.

Las mediciones permiten justificar qué ha mejorado.

Próxima sesión: describiremos el sistema completo con Docker Compose.